

Crafter-in-Cogniland

A partially-observable navigation env with a one-shot build commitment

Abstract

CRAFTER-IN-COGNILAND is a small POMDP navigation environment built on Crafter-style procedural terrain. An agent must cross from a spawn bubble to a target flag, where the terrain places a *barrier* (a lake or a rocky ridge) on the direct path. The agent may **commit once** to building a *raft* (cancels water slip) or a *harness* (cancels rock slip); the choice is irreversible and the agent is never told which item it built. The task therefore couples geometric navigation with a single belief-driven commitment under partial observation. We train both a recurrent PPO baseline and a pure-JAX DreamerV3 world model against a shared metric schema.

1 Environment at a glance

Observation An egocentric tile-id crop plus a few scalars. The two trainers consume *slightly different* observations:

$$\underbrace{o_{\text{PPO}} = \{ \text{semantic}_{V \times V}, \text{skill_active} \in \{0, 1, 2\} \}}_{\text{PyTorch env, } V=21} \quad \underbrace{o_{\text{Dreamer}} = \{ \text{minimap}_{V \times V}, \text{scalars} \in \mathbb{R}^4 \}}_{\text{JAX env}}.$$

Both crops are the $V \times V$ window of tile ids centred on the agent, with off-map cells padded by the `oob` id (6). The PPO env’s `skill_active` scalar names *which* item is active (0=none, 1=harness, 2=raft), so the agent need not remember its own build. What it *must* still infer is the hidden *map identity* (lake vs. rocky) — the local crop never labels the biome. Dreamer’s `scalars` additionally supply `[compassr, compassc, build_active, t/tmax]`: a unit vector toward the target and a normalised step counter. PPO receives *neither* compass nor step count — it must read heading from the crop alone.

Action A single discrete action with two terminal build choices:

$$a \in \{\text{up, down, left, right, build_raft, build_harness}\}.$$

Selecting `build_raft` or `build_harness` commits that item once and for all; later builds are no-ops (still charged a step). The model also emits a continuous `inference_scalar` $\in [-1, 1]$ (the belief head, §3), but that is an internal map-recognition probe, *not* an action consumed by the environment.

Reward Flat slack plus potential-based shaping toward the target:

$$r_t = \underbrace{-0.02}_{\text{slack}} + \underbrace{0.01 (\text{ctg}_{t-1} - \text{ctg}_t)}_{\text{PBRs, in cells}}, \quad \text{reach bonus} = 0.$$

`ctg` is the unit-cost Dijkstra cost-to-go. With the sparse reach bonus disabled, the shaping term is the sole positive signal, so returns keep improving as the route tightens (no value cliff at the goal).

Slip Every tile except `lava/oob` is walkable. The skill is purely a *slip-rate* modifier, and each skill owns *two* terrains: **raft** zeroes **water** and **sand**, **harness** zeroes **dirt** and **rock**. Stepping onto a tile fails (agent stays put) with its base slip unless the matching skill is active: the major barriers **water/rock** slip 0.75, the minor terrains **sand/dirt** slip 0.30. **grass/target** never slip; **tree** always slips 0.75. So the matching skill clears both its barrier *and* the matching apron, while the wrong skill helps neither.

Sprite	Name	ID	Walkable	Slip (none / raft / harness)	Role
	grass	0	yes	0 / 0 / 0	spawn-bubble & open ground
	dirt	1	yes	0.30 / 0.30 / 0	harness apron (rock)
	sand	2	yes	0.30 / 0 / 0.30	raft apron (shoreline)
	water	3	yes	0.75 / 0 / 0.75	raft barrier
	rock	4	yes	0.75 / 0.75 / 0	harness barrier
	target	5	yes	0 / 0 / 0	goal cell (flag)
	oob	6	no	—	off-map padding seen in the crop
	tree	7	yes	0.75 / 0.75 / 0.75	always-slips obstacle on grass
	lava	8	no	—	impassable hazard in rock regions

Table 1: Tile vocabulary and the slip mechanic (slip columns: none / raft / harness). Each skill zeroes *two* terrains (**bold**): raft → **water**+**sand**, harness → **dirt**+**rock**. Major barriers slip 0.75, minor aprons 0.30; **grass/target** never slip.

2 Procedural maps: a compositional generator

Each map is *composed* in three layers rather than thresholded from raw noise. **(1) Grass base.** **(2) Component layer:** a small vocabulary of analytic terrain *atoms* (Fig. 1) — a bent, hourglass-thick *ridge* and an irregular *round mountain* (both **rock**); a thin meandering *river* and a lobed *lake* (both **water**). Each atom is a closed-form field, so placing it is just a rotation + translation followed by a threshold into a binary mask. **(3) Texture layer:** independent simplex fields sprinkle clustered **tree** forest and sparse **dirt/sand** over the open grass, and a collar of **dirt** (around ridges) or **sand** (around water) is grown a few cells out from each barrier.

The agent travels from a spawn (bottom-left) to a target (top-right), so barriers are placed to straddle that diagonal — and the target/spawn keep a small grass clearance, so a barrier may pass *nearby* the target but never cover it. The three **biomes** differ in which atoms appear and how they are placed:

- **rocky:** a *ridge* with one end pinned to the top-right (target) corner, running toward the lower-left, plus a non-overlapping *lake*; the ridge wins any overlap ⇒ *harness* is the cheap route.
- **lake:** a corner-anchored *river* plus a non-overlapping *round mountain*; the river wins any overlap ⇒ *raft*.

- **balanced**: the map is split along the anti-diagonal — one half gets a water atom, the other a rock atom, with extra forest down the middle grass corridor $\Rightarrow$ building *nothing* is usually best.

Because the env is a POMDP, maps are *not* re-sampled to enforce a strict per-skill cost ordering (as the old simplex pipeline was); the only feasibility guard is that the target stays reachable.

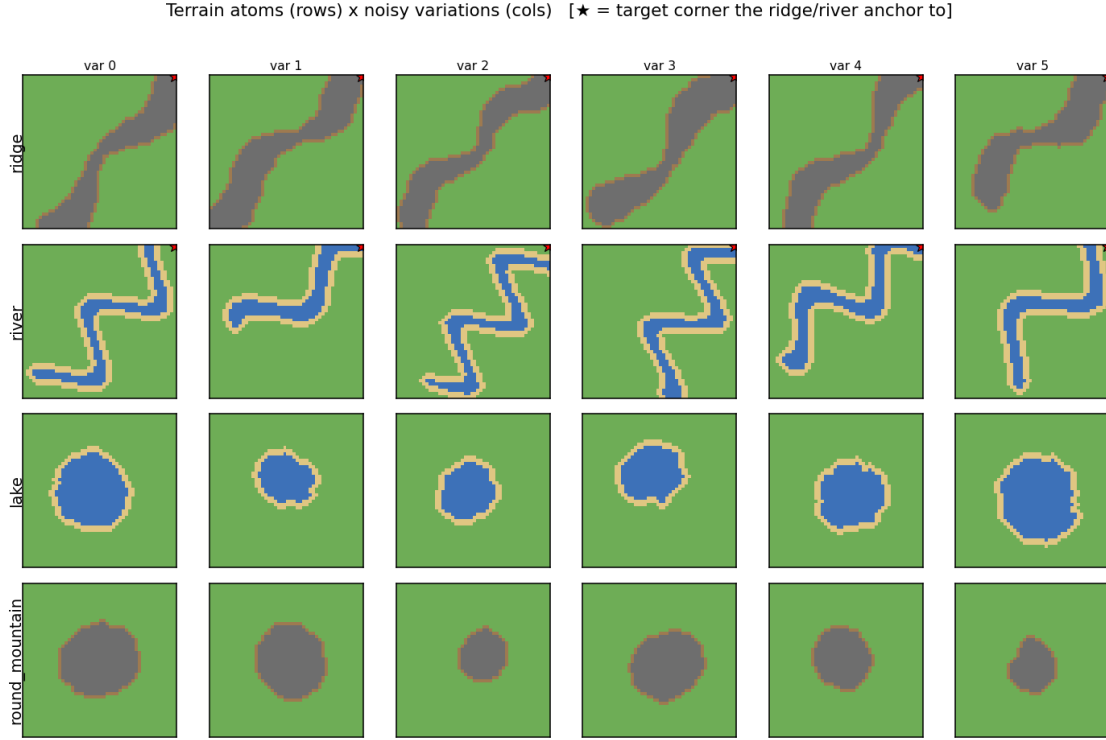


Figure 1: Terrain atoms (rows) $\times$ noisy variations (columns). Every atom is an analytic field placed by rotation/translation; **ridge** and **river** are *corner-anchored* — one end pinned to the target (★) corner — while **lake** and **round_mountain** are free-floating blobs. High-frequency simplex noise ragged-ifies each boundary; water gets a **sand** beach, rock a **dirt** skirt.

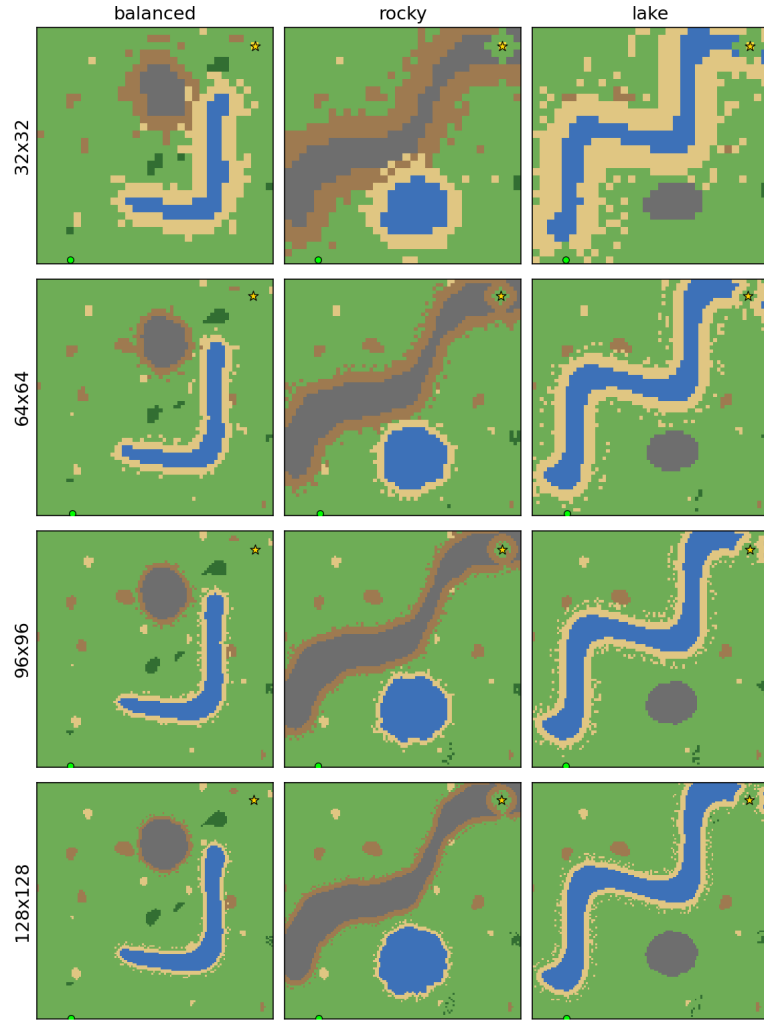


Figure 2: Map scale $\times$ biome. Rows are side length (32/64/96/128); columns are biome (balanced, rocky, lake). Spawn (●) sits bottom-left, target (★) top-right. Blue is water, grey is rock; balanced keeps both scarce.

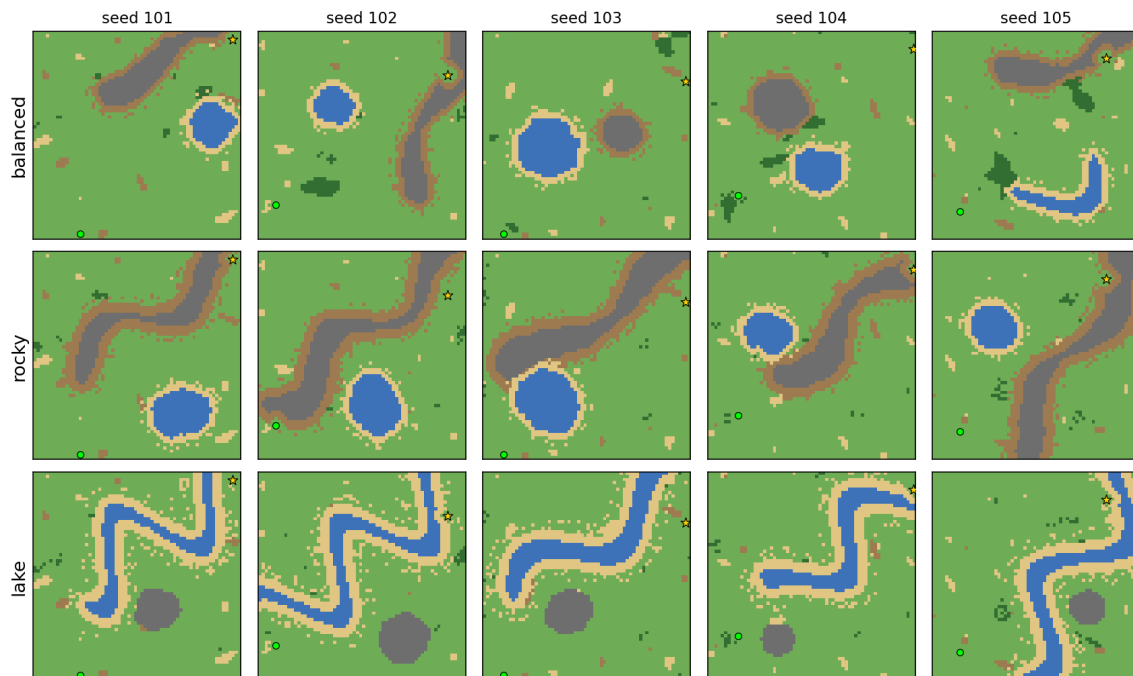


Figure 3: Seed diversity at 64×64 . Rows are biome (**balanced/rocky/lake**); columns are different generation seeds. Each seed yields fresh barrier geometry, so within a biome the agent cannot memorise a layout — it must read the local terrain online.

3 Agents

3.1 Recurrent PPO baseline (the released checkpoints)

The PPO agent is a tile-embedding CNN $\rightarrow$ GRU recurrent policy (Fig. 4). The $V \times V$ tile crop is embedded (16-d) and augmented with two CoordConv channels, passed through three 3×3 convs, flattened, concatenated with `skill_active`, and projected to a 256-d embedding. A 128-unit GRU carries memory across the episode (so the agent can recall its committed item). Three heads read the GRU state.

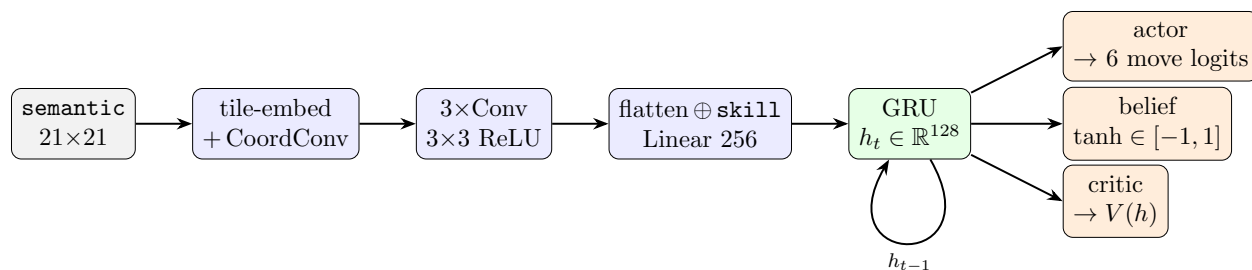


Figure 4: PPO-GRU policy. The *belief* head outputs the `inference_scalar`, a deterministic tanh map-recognition probe; it is *not* consumed by the env (the build is a discrete action). The actor now has 6 move logits (incl. `buildRAFT`, `buildHARNESS`).

The inference scalar, and how it is trained. The `inference_scalar` is produced by the *belief* head: a single linear unit squashed by tanh into $[-1, 1]$. It is *deterministic* (unlike the move head,

it is not sampled), so +1 leans lake/raft, -1 leans rocky/harness, 0 is undecided. It is *not* an action — the env never reads it; the build is committed by the discrete `build_raft` / `build_harness` moves instead. The scalar is purely an internal map-recognition probe that pressures the GRU state to encode the hidden biome. During training it gets a privileged **supervised auxiliary signal**: an MSE loss to the target $b^* = \{\text{lake} : +1, \text{rocky} : -1, \text{balanced} : 0\}$ derived from the map label. The total objective is

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{PPO}}^{\text{clip}}}_{\text{move policy}} + 0.5 \underbrace{(V - \hat{R})^2}_{\text{value}} + 0.5 \underbrace{(\tanh(b) - b^*)^2}_{\text{belief MSE}} - 0.01 \mathcal{H}[\pi_{\text{move}}].$$

Each term plays a distinct role, and the gradients are summed because the four outputs share one trunk (CNN $\rightarrow$ GRU) but answer four different questions:

- $\mathcal{L}_{\text{PPO}}^{\text{clip}}$ — the **actor** loss. The clipped surrogate raises the probability of moves whose advantage $\hat{A}$ is positive while the ratio π/π_{old} is clamped to $[1-0.2, 1+0.2]$, which bounds the policy step per update and keeps the on-policy approximation valid. This is the only term that learns *where to move*.
- $0.5(V - \hat{R})^2$ — the **critic** loss. Regressing the value head onto the GAE return $\hat{R}$ gives the baseline that makes the advantage estimate low-variance; without an accurate V the actor’s $\hat{A}$ would be noisy and learning would stall. Weight 0.5 is the standard PPO value coefficient.
- $0.5(\tanh(b) - b^*)^2$ — the **belief** loss. Move reward alone teaches the build choice only through a long, noisy credit-assignment path (build now $\rightarrow$ less slip many steps later). Supervising the belief head directly against the privileged label b^* turns *which item to build* into a cheap one-step regression, so the commitment is learned reliably; at test time the head still has to infer b from the crop, so no privileged information leaks into control.
- $-0.01 \mathcal{H}[\pi_{\text{move}}]$ — the **entropy bonus**. Subtracting (a small multiple of) the move-policy entropy rewards keeping the action distribution spread out, sustaining exploration and stopping the policy from collapsing to a deterministic route before it has seen enough terrain.

Optimiser Adam ($\text{lr} = 3 \times 10^{-4}$), GAE ($\gamma = 0.99$, $\lambda = 0.95$), clip 0.2, 128 steps $\times$ 4 epochs $\times$ 4 minibatches. The four released checkpoints (`ppo_gru_size\{32,64,96,128\}`) are each trained for 5M env steps on `map_type=random` at *one fixed map size*.

3.2 DreamerV3 world model (high level)

The Dreamer trainer learns a recurrent state-space model (RSSM) and trains the actor-critic purely *in imagination* (Fig. 5). Because the minimap is symbolic, the encoder/decoder are MLPs (RMSNorm + SiLU) over the flat $V \cdot V + 4$ observation vector (minimap $\oplus$ the 4 scalars) — no pixel CNN.

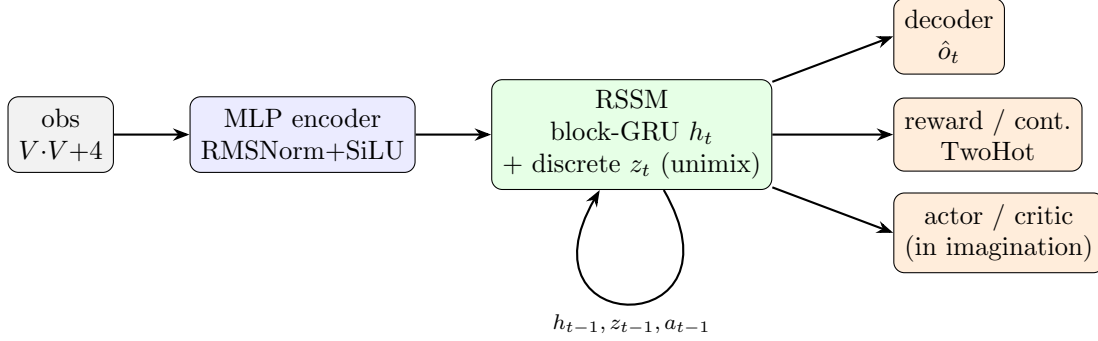


Figure 5: High-level DreamerV3. The RSSM rolls forward in latent space; the actor-critic is optimised on imagined trajectories with λ -returns and a slow (EMA) critic.

Component	Choice
Latent	deterministic h (block GRU) + discrete stochastic z , unimix 1%
Reward / value	TwoHot over 255 symlog bins
Slow critic	EMA decay 0.98 + cross-entropy reg
Actor scale	RetNorm $S = \max(1, \text{Per}_{95} - \text{Per}_5)$
KL loss	balanced dyn/rep, free-nats 1, $\beta_{\text{rep}}=0.1$
Optimiser	LaProp ($\varepsilon=10^{-20}$) + AGC 0.3
Defaults	$\gamma=0.997$, $\lambda=0.95$, entropy 3×10^{-4} , lr 4×10^{-5}

Table 2: DreamerV3 components (paper-aligned). The discrete build action folds the PPO belief scalar into two terminal actions (build_raft, build_harness).

4 Behaviour: 100 stochastic rollouts per map

Figure 6 probes each PPO-GRU checkpoint on a *single fixed map* (identical spawn and target) per biome at the model’s trained size. We sample the move policy 100 times — so divergence between paths reflects both policy stochasticity and terrain slip — and overlay the trajectories as thin lines over a darkened map. Crucially, **each segment is coloured by the agent’s currently-active skill**: dark blue while it carries nothing, orange once it has built a raft, yellow once it has built a harness. A path that starts blue and turns orange/yellow part-way thus shows exactly *when* and *what* the agent committed to.

The bottom row adds the **skill-choice matrix** (the same quantity logged to W&B): for each model, the fraction of the 100 trials per biome that ended with each skill (rows **balanced/rocky/lake**, columns **noskill/harness/raft**). A competent agent is near-diagonal: **lake**→**raft**, **rocky**→**harness**, **balanced**→**noskill**.

The full set of grids (both slip regimes, five map seeds each) lives in **figures/trajectory_grids/**; Fig. 6 shows the slip = 0.75 models on seed 42. Across seeds the **lake** and **rocky** rows saturate on the correct skill, while **balanced** stays noisier — the hardest case, since the agent must learn to build *nothing*.

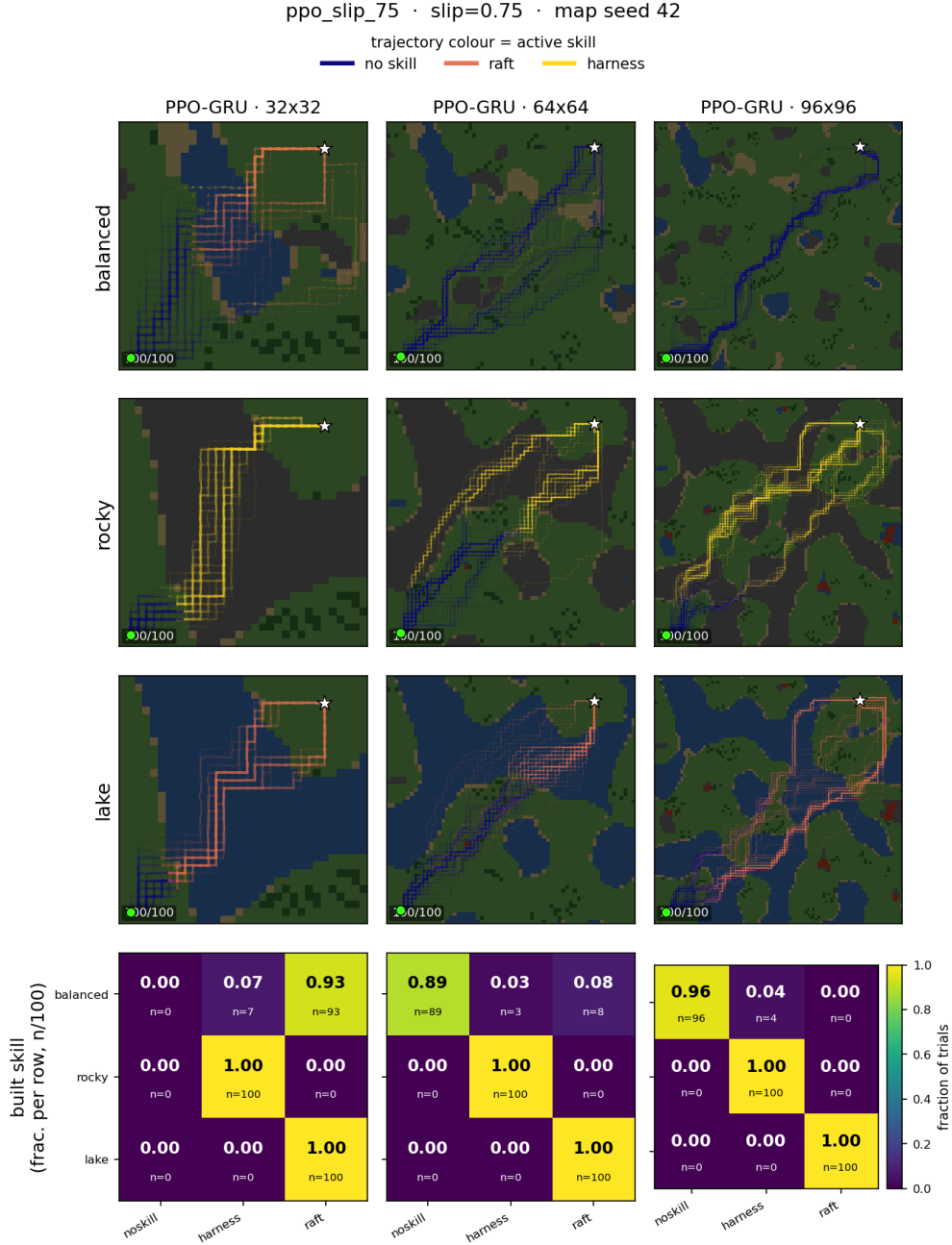


Figure 6: Trajectory + skill grid (slip= 0.75, map seed 42). Rows are biome (balanced/rocky/lake); columns are the PPO-GRU checkpoint, each on the map size it was trained on (32/64/96; the 128 model is omitted). Trajectory colour = active skill (no skill = dark blue, raft = orange, harness = yellow); spawn = ●, target = ★, inset = successes/100. Bottom row: skill-choice matrix, row-normalised fraction of trials (with raw counts), viridis scale at right.

4.1 Measuring path diversity

To say how *varied* a policy’s paths are, we score the 100 rollouts per map with two numbers. For each rollout we build its **occupancy**: the fraction of steps spent on each map cell (a probability map over cells).

Occupancy entropy (nats). Average the 100 occupancies into one map and take its Shannon entropy

$$H = - \sum_{\text{cells } c} p(c) \ln p(c) \quad (\text{nats}).$$

It measures *how much ground is used*: low when every run hugs the same short corridor, high when the policy spreads over many cells. Bounded by $\ln(\text{visited cells})$.

Across-trajectory JSD (nats). The Jensen–Shannon divergence among the 100 per-run occupancies,

$$\text{JSD} = H\left(\frac{1}{N} \sum_i o_i\right) - \frac{1}{N} \sum_i H(o_i) \geq 0,$$

i.e. the entropy of the average minus the average entropy. It measures *how much the runs differ from each other*: 0 when all paths are identical, larger when each run takes a different route. Bounded by $\ln N$.

The two are complementary: entropy can be high simply because a single wandering path covers many cells, whereas JSD isolates *between-run* spread. The **diverse** preset scores higher on both (occupancy entropy ≈ 6.5 nats, JSD ≈ 2.0) than the **efficient** preset (≈ 5.6 / ≈ 1.1), while both still reach the target — diversity bought without giving up the task. Both are computed by `cogniland.trajectory_variability` and logged to W&B.